

# 👉 XAI: Post-hoc - SmoothGrad & Integrated Gradients

FAMAF - UNC

First Semester 2026



**eXplainable  
Artificial Intelligence**

## Disclaimer ---

This course is based on

### **Explainable Artificial Intelligence**

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

<https://interpretable-ml-class.github.io/>

---

## SmoothGrad: removing noise by adding noise

---

Daniel Smilkov<sup>1</sup> Nikhil Thorat<sup>1</sup> Been Kim<sup>1</sup> Fernanda Viégas<sup>1</sup> Martin Wattenberg<sup>1</sup>

### Abstract

Explaining the output of a deep network remains a challenge. In the case of an image classifier, one type of explanation is to identify pixels that strongly influence the final decision. A starting point for this strategy is the gradient of the class score function with respect to the input image. This gradient can be interpreted as a *sensitivity map*, and there are several techniques that elaborate on this basic idea. This paper makes two contributions: it introduces SMOOTHGRAD, a simple method that can help visually sharpen gradient-based sensitivity maps, and it discusses lessons in the visualization of these maps. We publish the code for our experiments and a website with our results.

A common approach to understanding the decisions of image classification systems is to find regions of an image that were particularly influential to the final classification. (Baehrens et al., 2010; Zeiler & Fergus, 2014; Springenberg et al., 2014; Zhou et al., 2016; Selvaraju et al., 2016; Sundararajan et al., 2017; Zintgraf et al., 2016). These approaches (variously called sensitivity maps, saliency maps, or pixel attribution maps; see discussion in Section 2; use occlusion techniques or calculations with gradients to assign an “importance” value to individual pixels which is meant to reflect their influence on the final classification.

In practice these techniques often do seem to highlight regions that can be meaningful to humans, such as the eyes in a face. At the same time, sensitivity maps are often visually noisy, highlighting some pixels that—to a human eye—seem randomly selected. Of course, a priori we cannot determine if this noise reflects an underlying truth about how

(Smilkov et al. 2017)

## 🍷 Motivation ---

We want **post-hoc explanations of image classifiers**

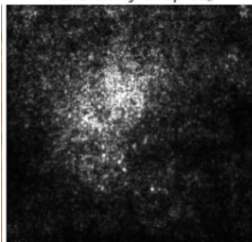
### *Solution:*

- **Sensitivity maps** (a.k.a. saliency maps, pixel attribution maps)
- Visual interpretation of gradient of class activation function w.r.t input image
  - ▶ Structured as grayscale image w/ dimension same as input image
  - ▶ Brightness of pixel  $\propto$  importance to classification decision

Image



Sensitivity map  $M_c$





## 🍷 Background: Gradients as Sensitivity Maps ---

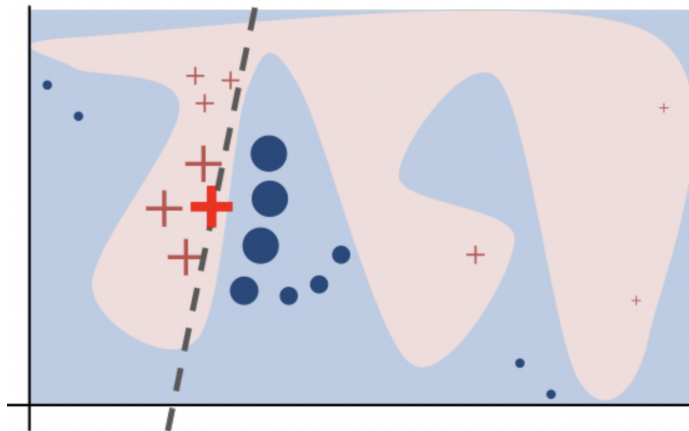
- $N$ : network that classifies images into one class from set  $C$
- Given input image  $x$ ,  $N$  typically computes class activation function  $S_c$  for each  $c \in C$
- Final classification  $\text{class}(x) = \arg\max_{c \in C} S_c(x)$
- **Sensitivity map given by**

$$M_c(x) = \partial S_c(x) / \partial x$$

- **Intuition:**  $M_c$  represents how much difference a tiny change in each pixel of  $x$  would make to the classification score for class  $c$ .
- In practice, this is done with **backpropagation** — same as during training, but instead of updating weights, we stop at the input pixels. The result is a gradient map with one value per pixel.

## 🍷 Related Work: Perturbation Methods ---

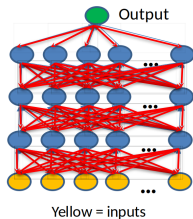
- **Key idea:** generate a perturbed dataset to fit an explainable model
  - ▶ LIME
  - ▶ KernelSHAP



## 🍷 Related Work: Backpropagation ---

Smoothgrad lives in the family of backpropagation-based methods for explaining neural networks.

All methods propagate a signal backwards from the output to the inputs to assign importance scores to pixels — just like training, but stopping at the input. They differ in **how** they modify that signal along the way:



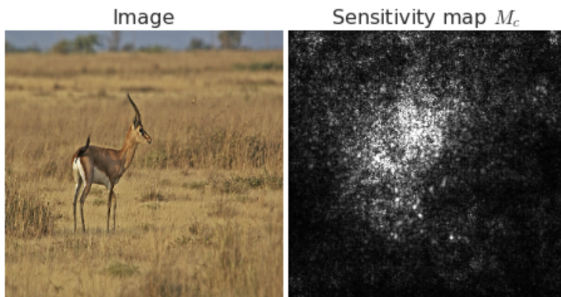
- **Vanilla gradients** — raw gradient  $\partial S_c / \partial x$ , no modifications. Baseline for SmoothGrad.
- **Integrated Gradients** — averages gradients along a path from a reference image to  $x$ .
- **DeepLIFT** — propagates activation *differences* relative to a reference, instead of raw gradients.
- **LRP** — redistributes output relevance backwards layer by layer using conservation rules.
- **Guided Backprop / Deconvolution** — discards negative gradients through ReLUs to highlight only positive contributions.

**SmoothGrad is complementary**, not a replacement — it can be applied on top of any of these to reduce visual noise.

## 🍷 Limitations of Sensitivity Maps ---

### Visually noisy

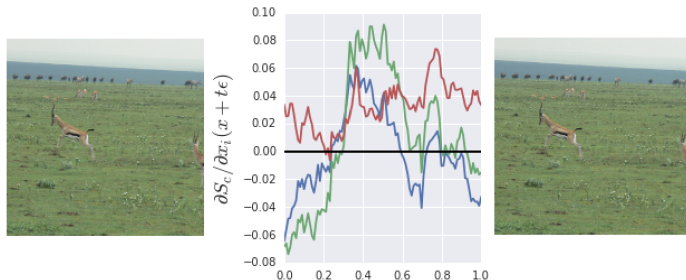
- Often highlight pixels that—to a human eye—seem randomly selected
- *a priori*, we cannot know if this noise reflects an underlying truth about how networks perform classification, or is due to more superficial factors
- This is why saliency maps are typically visualized as a heatmap-like plot.



## 🍷 Theory Behind SmoothGrad: Noisy Gradients ---

**Key idea:** Noisy maps are due to noisy gradients

- The derivative of  $S_c$  may fluctuate sharply at small scales — networks use **ReLU**s, so  $S_c$  is **not even continuously differentiable**.
- *Two images that look identical to a human can have very different gradients — so the raw gradient at a single point is not a reliable importance signal.*
- The noise you see in the map is not necessarily telling you something meaningful it may just be an accident of the mathematical landscape of  $S_c$



## SmoothGrad: Intuition ---

### Simple solution

- take an image of interest
- sample similar images by adding Gaussian noise to the image
- take the average of the resulting sensitivity maps for each sampled image
- This smoothes the gradient

## SmoothGrad: Algorithm ---

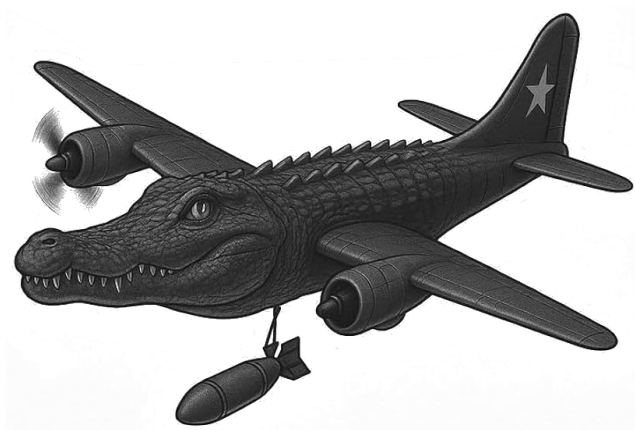
- 1 Take random samples in a neighborhood of an input  $x$  with added noise
- 2 Average the resulting sensitivity maps

$$\hat{M}_c(x) = \frac{1}{n} \sum_1^n M_c(x + \mathcal{N}(0, \sigma^2))$$

- $n$  is the number of samples,
- $\mathcal{N}(0, \sigma^2)$  represents Gaussian noise with standard deviation  $\sigma$ .

## 🍷 Experimental Setup ---

- Performed SmoothGrad on visualizations of two neural networks:
  - ▶ Inception v3 model by Google that was trained on the ILSVRC-2013 dataset
  - ▶ Convolutional MNIST model based on the TensorFlow tutorial





## 🍷 Choosing Hyperparameters ( $\sigma$ : std. dev.) ---

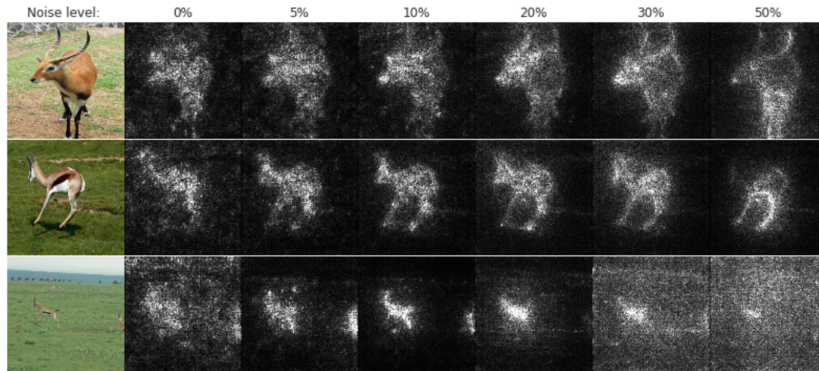


Figure 3. Effect of noise level (columns) on our method for 5 images of the gazelle class in ImageNet (rows). Each sensitivity map is obtained by applying Gaussian noise  $\mathcal{N}(0, \sigma^2)$  to the input pixels for 50 samples, and averaging them. The noise level corresponds to  $\sigma/(x_{max} - x_{min})$ .

$$\hat{M}_c(x) = \frac{1}{n} \sum_1^n M_c(x + \mathcal{N}(0, \sigma^2))$$

$\sigma$ : the standard deviation of the Gaussian noise

## 🍷 Choosing Hyperparameters ( $n$ : sample size) ---

$$\hat{M}_c(x) = \frac{1}{n} \sum_1^n M_c(x + \mathcal{N}(0, \sigma^2))$$

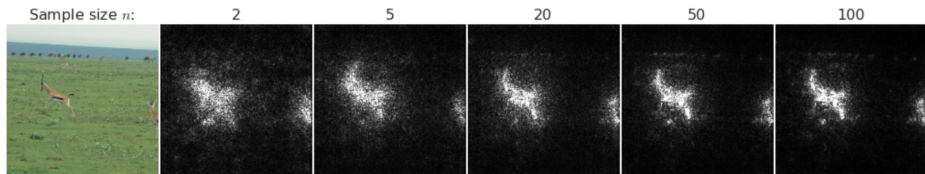


Figure 4. Effect of sample size on the estimated gradient for inception. 10% noise was applied to each image.

# 🍷 Qualitative Results: Visualization Techniques ---

## ● Absolute Value of Gradients

- ▶ depends on the characteristics of dataset



## ● Capping outlying values

- ▶ presence of few pixels that have much higher gradients than the average
- ▶ capping to 99 percentile

## ● Multiplying maps with images

- ▶ produce simpler & sharper images (Shrikumar et al., 2017; Sundararajan et al., 2017)
- ▶ Downside: Pixels with values of 0 will never show up on the sensitivity map.
- ▶ Upside: when viewing the importance of the feature as contribution to the image

## 🍷 Qualitative Results: Visual Coherence 1/2 ---

### Definition

**Visual Coherence** Highlights are only on the object of interest, not the background

Comparison with three gradient-based methods

- Vanilla gradient
- Integrated Gradients
- Guided BackProp



## Qualitative Results: Visual Coherence 2/2 ---

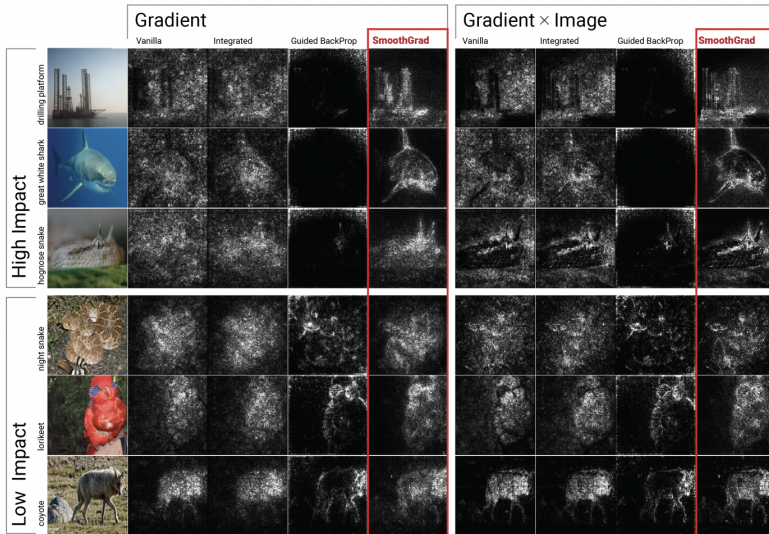


Figure 5. Qualitative evaluation of different methods. First three (last three) rows show examples where applying SMOOTHGRAD had high (low) impact on the quality of sensitivity map.

## 🍷 Qualitative Results: Discriminativity ---

### Definition

**Discriminativity** the ability to explain / distinguish separate objects without confusion

### Open Problem

Which properties affect the discriminativity of a given methods?

- Why did GBP show the worst performance?

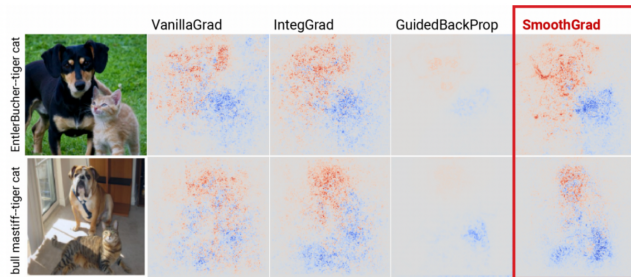
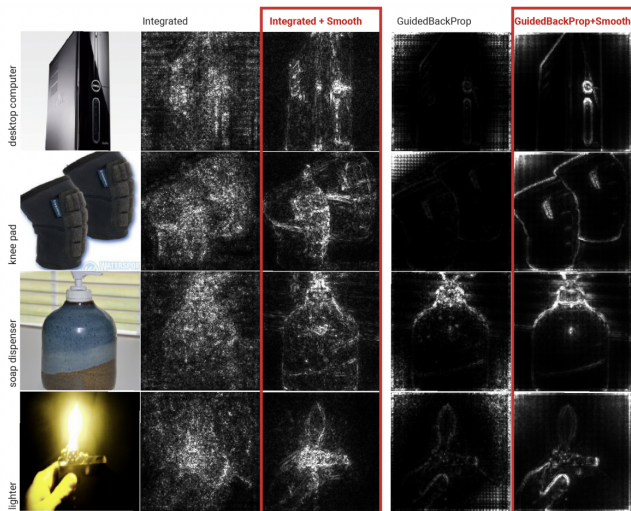


Figure 6. Discriminativity of different methods. For each image, we visualize the difference  $\text{scale}(\partial y_1 / \partial x) - \text{scale}(\partial y_2 / \partial x)$  where  $y_1$  and  $y_2$  are the logits for the first and the second class (i.e., cat or dog) and  $\text{scale}()$  normalizes the gradient values to be between  $[0, 1]$ . The values are plotted using a diverging color map  $[-1, 0, 1] \mapsto [\text{blue}, \text{gray}, \text{red}]$ . Each method is represented in columns.

## 🍷 Combining with Other Methods ---

The same smoothing procedure can be used to augment any gradient-based method.



## 🍷 Limitations / Discussion ---

- Completely qualitative results, can we get **quantitative metrics**?
- Noisy sensitivity maps are **due to noisy gradients**
  - ▶ Is this true?
  - ▶ Future work: look for further evidence and theoretical arguments
- Does SmoothGrad **generalize** to other networks & tasks?
- How do we tradeoff between making picture pretty and being faithful to the model? Do you think SmoothGrad handled this tradeoff well?



---

## Axiomatic Attribution for Deep Networks

---

Mukund Sundararajan<sup>\*1</sup> Ankur Taly<sup>\*1</sup> Qiqi Yan<sup>\*1</sup>

### Abstract

We study the problem of attributing the prediction of a deep network to its input features, a problem previously studied by several other works. We identify two fundamental axioms—*Sensitivity* and *Implementation Invariance* that attribution methods ought to satisfy. We show that they are not satisfied by most known attribution methods, which we consider to be a fundamental weakness of those methods. We use the axioms to guide the design of a new attribution method called *Integrated Gradients*. Our method requires no modification to the original

Shrikumar et al., 2016; Binder et al., 2016; Springenberg et al., 2014).

The intention of these works is to understand the input-output behavior of the deep network, which gives us the ability to improve it. Such understandability is critical to all computer programs, including machine learning models. There are also other applications of attribution. They could be used within a product driven by machine learning to provide a rationale for the recommendation. For instance, a deep network that predicts a condition based on imaging could help inform the doctor of the part of the image that resulted in the recommendation. This could help the doctor understand the strengths and weaknesses of a model and components for it. We give such an example in

(Sundararajan, Taly, and Yan 2017)

## 🍷 Intuition 1/2 ---

How much happier would an extra \$1 make you... if you already have a billion dollars?



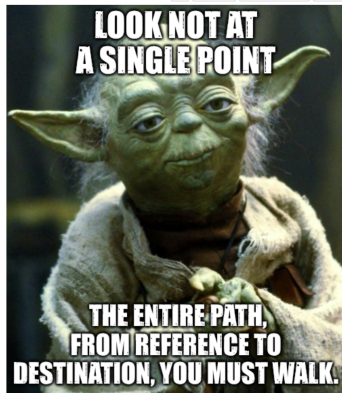
- The answer is “almost nothing” — but that doesn’t mean money is unimportant.
- This is **exactly the problem** with vanilla gradients:

If the model is **already very confident**, the score function is flat at that point, and the gradient is near zero for all pixels — even the obviously important ones.

- The gradient is a local measure — it tells you how the function is changing right there. But if you’ve already reached a flat region, that local information is misleading.

## 🍷 Intuition 2/2 ---

**Integrated Gradients** solves this with the same intuition you'd use in optimization to escape flat-regions:



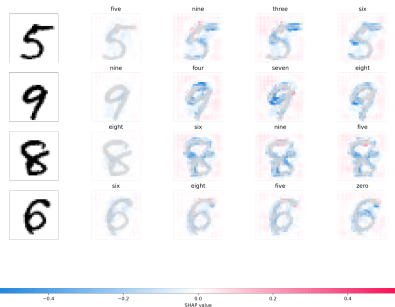
This way you capture the global contribution of each feature, not just its local behavior.

# Motivation and Problem Statement 1/2 ---

## Definition

### Feature Attribution

- $F : \mathbb{R}^n \rightarrow [0, 1]$  — deep network with  $n$  input features
  - $x'$  — baseline input representing "absence of signal"
  - $A_F(x, x') = (a_1, \dots, a_n)$  — attribution vector, same size as input
  - $a_i$  — contribution of feature  $x_i$  to the prediction  $F(x)$  relative to  $x'$
- 
- $F$  takes an image  $x$  with  $n$  pixels and returns a score between 0 and 1
  - An **attribution** is a vector of the same size as  $x$
  - Each  $a_i$  tells you how much pixel  $x_i$  contributed to the final prediction
  - Everything is measured relative to a reference image  $x'$  — the **baseline**
  - In other words: you distribute the output score among the input pixels, giving each one its “share of the blame”





## Motivation and Problem Statement 2/2 ---

- **Feature attribution** is a vector of the same size as the input — each  $a_i$  tells you how much feature  $x_i$  contributed to the prediction  $F(x)$ 
  - ▶ Ex. in a CNN, it reveals which pixels were responsible for a certain label being picked
  - ▶ LIME and SHAP are the same idea.
- **Problem:** attribution techniques are hard to evaluate empirically — hard to separate errors from the model vs errors from the attribution method
  - ▶ **Solution: axiomatic approach** — define mathematical properties that every attribution method *should* satisfy
  - ▶ Ex. Vanilla gradients *seem* reasonable but violate one of these axioms (Sensitivity) — provably, without needing experiments
- **Baseline**  $x'$ : a design choice — a reference input representing “absence of signal”
  - ▶ Attributions are always measured *relative* to it
  - ▶ Ex. black image for vision, zero embedding vector for text
  - ▶ A bad baseline leads to misleading attributions, even if the method is correct

## 🍷 Summary of Contributions ---

The paper proposes an **axiomatic approach**: instead of evaluating attribution methods empirically, define mathematical properties they *must* satisfy.

- **Sensitivity**: if a feature clearly changes the prediction, its attribution cannot be zero — otherwise the method is lying. Most methods (including vanilla gradients) violate this.
- **Implementation Invariance**: two networks that compute the same function should give the same attributions — attributions should not depend on irrelevant implementation details. Methods like DeepLIFT and LRP violate this.

These 2 axioms guide the design of **Integrated Gradients**: average the gradients along the entire path from the baseline  $x'$  to the input  $x$  — satisfies both axioms, no network modification needed.

# 🍷 Why Vanilla Gradients Fail: Violating Sensitivity

## 1/2 ---

**Key idea:** use  $\partial F / \partial x$  as a proxy for feature importance.

**Problem — breaks Sensitivity:** the prediction function can flatten at the input, giving zero gradient even when the feature clearly matters.

**Example:** single ReLU network  $f(x) = 1 - \text{ReLU}(1 - x)$ , baseline  $x = 0$ , input  $x = 2$ :

# 🍷 Why Vanilla Gradients Fail: Violating Sensitivity

1/1 ---

**Example:**  $f(x) = 1 - \text{ReLU}(1 - x)$ , baseline  $x = 0$ , input  $x = 2$

**Step 1: ReLU recap**  $\text{ReLU}(z) = \max(0, z)$  — returns  $z$  if positive, 0 if negative.

**Step 2: Evaluate at baseline  $x = 0$**  -  $1 - x = 1 - 0 = 1$ ,  $\text{ReLU}(1) = 1$ ,  $f(0) = 1 - 1 = 0$

**Step 3: Evaluate at input  $x = 2$**  -  $1 - x = 1 - 2 = -1$ ,  $\text{ReLU}(-1) = 0$ ,  $f(2) = 1 - 0 = 1$

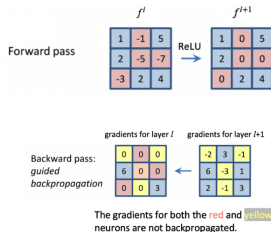
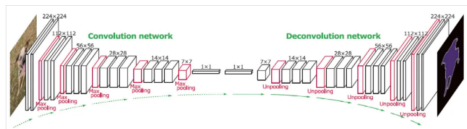
**Step 4: Gradient at  $x = 2$**  - For any  $x > 1$ :  $\text{ReLU}(1 - x) = 0$  always —  $f(x) = 1$ , flat, derivative = 0

Sensitivity violated

The prediction changed from 0 to 1, but the gradient says  $x$  is irrelevant



# 🍷 DeConvNets, Guided back-propagation (GBP) Also Break Sensitivity ---



- **GBP Rule:** only backpropagate through a ReLU if it was **on** at the input.
- **Forward pass:** ReLU zeros out negative activations — red/yellow cells in  $f^l$  become 0 in  $f^{l+1}$ .
- **Backward pass:** gradients are only propagated through ReLUs that were active at the input. Neurons zeroed out in the forward pass receive attribution 0 — even if their gradients in  $f^{l+1}$  are non-zero.

## Same root cause as vanilla gradients

- The method only looks at the current input, not the full path from  $x'$  to  $x$ .
- A feature that was off at the input gets attribution 0 — even if it clearly mattered for the prediction.

## 🍷 Other Attribution Methods: Methods that Break Implementation Invariance ---

### Methods that Break Implementation Invariance

- DeepLift and Layer-wise relevance propagation (LRP)



# The Method

# Integrated Gradients ---

## Definition

The **path integral** of the gradients along the **straight-line path** from the baseline  $x'$  to the input  $x$ .

$$\text{IntegratedGrads}_i(x) := (x_i - x'_i) \times \int_{\alpha=0}^1 \frac{\partial F(x' + \alpha \times (x - x'))}{\partial x_i} d\alpha$$

## New Axiom

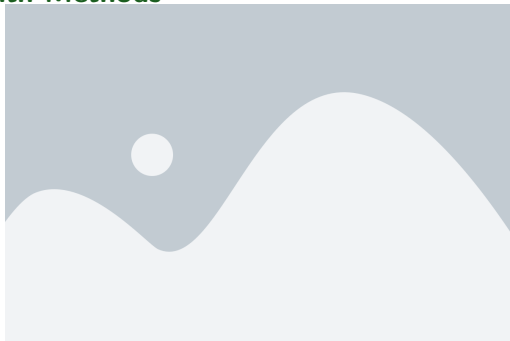
**Completeness:** The sum of the attributions is equal to the difference of the outputs.

**Proposition 1.** If  $F : \mathbb{R}^n \rightarrow \mathbb{R}$  is differentiable almost everywhere<sup>1</sup> then

$$\sum_{i=1}^n \text{IntegratedGrads}_i(x) = F(x) - F(x')$$

# 🍷 Uniqueness of Integrated Gradients ---

## Path Methods



$\text{PathIntegratedGrads}_i^\gamma(x) :=$

$$\int_{\alpha=0}^1 \frac{\partial F(\gamma(\alpha))}{\partial \gamma_i(\alpha)} \frac{\partial \gamma_i(\alpha)}{\partial \alpha} d\alpha$$

## Axioms

- **Sensitivity (b):** If the function does not depend (mathematically) on some input, then the attribution for that input is always zero. -
- **Linearity:** Attributions preserve any linearity within the network.

$$a \times f_1 + b \times f_2$$

- **Symmetry-Preserving:** For symmetric variables, if they have identical values in the input and identical values in the baseline, they then receive identical attributions.

$$\text{Si } F(x, y) = F(y, x)$$

# 🍷 Using Integrated Gradients ---

## Selecting a Baseline

**\*\*Two Components:\*\***

- Zero-Score:  $F(x') \approx 0$  - Conveys Absence of Signal

**\*\*Examples:\*\***

- Object Recognition: All-black image - Text: All-zero input embedding vector

## Computing IGs

$\text{IntegratedGrads}_i^{\text{approx}}(x) :=$

$$(x_i - x'_i) \times \sum_{k=1}^m \frac{\partial F(x' + \frac{k}{m} \times (x - x'))}{\partial x_i} \times \frac{1}{m}$$

# Experimental Results

## 🍷 Object Recognition CNN ---

**Task:** Given image, predict the category of the object





## 🍷 Question Classification CNN ---

**Task:** Given question, predict what type of answer it is looking for.



## 🍷 Machine Translation RNN ---

**Task:** Given English sentence, predict German translation



## 🍷 Ligand Screening Graph CNN ---

**Task:** Given molecular graph, predict whether it is active against an enzyme



# 🍷 Conclusion and Discussion ---

## Summary

- Formalizes two axioms for attribution: **sensitivity, implementation invariance**
- Propose **integrated gradients** and argue that it is theoretically superior to other gradient-based methods (e.g. DeepLift, LRP, guided backprop, etc.)
- Perform experiments across several domains to showcase method

## Discussion Questions

- Are you convinced that these axioms are desirable?
- Do you see any strengths or weaknesses in the idea of producing explanations through an integrated path?
- Have the experiments convinced you of the superiority of their method?

# Thank You!

## References --- |

- Smilkov, Daniel, Nikhil Thorat, Been Kim, Fernanda Viégas, and Martin Wattenberg. 2017. “SmoothGrad: Removing Noise by Adding Noise.” *arXiv Preprint arXiv:1706.03825*.
- Sundararajan, Mukund, Ankur Taly, and Qiqi Yan. 2017. “Axiomatic Attribution for Deep Networks.” In *International Conference on Machine Learning*, 3319–28. PMLR.